0079. Word Search

Medium | Category: Backtracking | LeetCode: https://leetcode.com/problems/wo

Problem

- Does word exist in the grid as a path of horizontally/vertically adjacent cells, each used at most once?

Intuition

- Try every cell as a start; DFS matching one character at a time.
- Temporarily mark the cell as visited (overwrite with '#') and restore it when backtracking.

Brute Force

- The backtracking search is the brute force; the optimisation is pruning (stop at the first mismatch, optional character-count precheck).

Optimal Approach

- For each (r, c): if dfs(r, c, 0) return true.
- Dfs: if i == len return true; out of bounds or board[r][c] != word[i] return false.
- Mark, recurse in 4 directions, restore.

Worked Example

```
board ABCE / SFCS / ADEE, word "ABCCED".

(0,0)A -> (0,1)B -> (0,2)C -> (1,2)C -> (2,2)E -> (2,1)D  -> found
```

Java Solution

```java
public class WordSearch {
    public boolean exist(char[][] board, String word) {
        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++)
                if (dfs(board, word, r, c, 0)) return true;
        return false;
    }

    private boolean dfs(char[][] b, String w, int r, int c, int i) {
        if (i == w.length()) return true;
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || b[r][c] != w.charAt(i)) return false;
        char saved = b[r][c];
        b[r][c] = '#';
        boolean found = dfs(b, w, r + 1, c, i + 1) || dfs(b, w, r - 1, c, i + 1)
                || dfs(b, w, r, c + 1, i + 1) || dfs(b, w, r, c - 1, i + 1);
        b[r][c] = saved;
        return found;
    }
}
```

Complexity

- Time: O(m n 3^L). Space: O(L) recursion.

Pitfalls

- Forgetting to restore the cell breaks later searches.